StoryVox: A Multilingual Story Generation and Speech System

LINGALA SINDHU
Roll No. 23K91A05D3
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
Hyderabad, India
[EMAIL]

PADIGE AJAY
Roll No. 23K91A05H8
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
Hyderabad, India
[EMAIL]

MAHEK ANSARI
Roll No. 23K91A05D8
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
Hyderabad, India
[EMAIL]

MAHESHWARAM HARI KRISHNA
Roll No. 23K91A05D9
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
Hyderabad, India
[EMAIL]

A TEJASWINI
[DESIGNATION]
(project guide)
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
Hyderabad, India
[EMAIL]

***Abstract*---**Reading aloud a story that was written on demand, in the listener's own language, needs a text generator and a speech synthesiser that are usually studied apart. StoryVox joins them in one locally runnable application: a story idea, a language, a style and a length go to a 0.6-billion-parameter language model, and the text is narrated by a per-language VITS speech model, with every result stored in a library and scored by a speech recogniser. Nothing in the system is trained; it is a cascade of pretrained models. We measured it on one CPU laptop. Narration in 12 languages ran at a mean real-time factor of 0.25 with a mean normalised character error rate of 0.088 after transcription. Story generation is the weak link: with the shipped prompt the model answered in the requested language for only 0.18 of 17 test cases, and four of four non-English stories were narrated by a foreign voice. The built-in evaluation also returns no error rates for story-length audio. We report these findings and the fixes they suggest.

***Index Terms*---**cascade pipeline, multilingual speech synthesis, real-time factor, speech recognition, story generation, text-to-speech evaluation, word error rate.

I. INTRODUCTION

Children's tales, language lessons and accessibility tools all benefit from stories that are read aloud, and large language models can now write a story from a one-line idea. Turning that text into speech in many languages is a separate problem: one line of work scales text-to-speech to hundreds of languages [2], [3], another builds expressive or child-directed voices for a few [4], [5], and a third translates text into speech directly [1]. A person who simply wants a story in Hindi or Spanish today has to run a language model, find a speech model for the language and check by ear whether the result is usable. Most of these systems also expect a GPU or large amounts of training data, which a school or a home laptop does not have. The problem addressed here is therefore practical rather than architectural: can a student-built application, using only pretrained parts and a CPU, generate a short story on request in a chosen language, read it aloud, and report honestly how well each step worked? The intended users are teachers, parents and learners who want a story on a topic of their choice, and the same pipeline could serve accessibility tools that read generated text aloud.

StoryVox closes that loop on a CPU. It is a web application over a FastAPI service that accepts a story idea (up to 2,000 characters), a language (15 offered), a style (8) and a length (3), generates the story with Qwen3-0.6B [6], narrates it with the language's MMS-TTS checkpoint [2] and saves story and audio so that they can be replayed and downloaded. An evaluation page transcribes the narration with Whisper [7] and reports word and character error rates, topic similarity, ROUGE-L and real-time factor.

Contributions: (1) StoryVox, a tested, locally runnable pipeline and console from story idea to saved narration. (2) A round-trip measurement of narration across 15 languages on the FLEURS benchmark [8], giving speed and intelligibility per language. (3) A controlled comparison of three story-generation conditions on 17 test cases that exposes a language-mismatch problem. (4) A list of defects found while measuring, among them an evaluation page that cannot score audio longer than 30 s, with a separate script that does. Section II reviews related work, Section III describes the method, Section IV gives results and Section V concludes.

A. System Hardware

Every number was measured on one machine: Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB RAM, Windows 11, no GPU, so all models ran on the CPU.

B. Software Requirements

Python 3.11.9, PyTorch 2.14.1 and Hugging Face Transformers 5.20.2 [9]; FastAPI with SQLite for the service, React 19 with Vite for the console, jiwer for error rates, sentence-transformers for similarity, and ffmpeg so that Whisper can read WAV files. The first run downloads the model weights; afterwards no data leave the machine.

II. LITERATURE SURVEY

Base paper. Tran and Sakti [1] (SAM Translator) translate source-language text into target speech without a cascade. A multilingual BART [10] encoder-decoder predicts discrete speech units from text and a Grad-TTS decoder turns them into a waveform. Three additions raise quality: self-paced learning [11] orders training examples from easy to hard, sparse mixture-of-experts layers [12] (8 experts, top-2 routing) raise capacity from 611M to 1.7B parameters, and data-augmented self-training adds synthetic pairs for low-

resource languages. Output speech is transcribed and scored with BLEU on CVSS-C (21 source languages to English) and CoVoST-2. Reported gains are about 2-3% relative for high-resource and 7-10% for low-resource languages on CVSS-C, and about 21.6% for Welsh with self-training. The authors note an English-only target, weaker results than a cascade for some languages, repetition errors and a heavy budget: an A100 GPU for 1.5-2 days.

Speech at scale. VITS [13] maps text directly to a waveform with a variational autoencoder, a normalising flow and adversarial training; MMS [2] released one such checkpoint for 1,107 languages. OmniVoice [3] covers 646 languages with a diffusion language model and reports a 4.00% average character error rate on FLEURS at RTF 0.0319 on a GPU; MultiGen [5] fine-tunes a language model for child-friendly Malay, Tamil and Mandarin; EmoSSLSphere [4] adds emotion control. **Stories.** TinyStories [14] shows that very small models write coherent short English stories when the data are narrow; hierarchical generation [15] and nucleus sampling [16] address plot and repetition; Qwen3 [6] is an open family whose 0.6B member fits a laptop; Kahaani [17] pairs story co-creation with speech for children. **Evaluation.** Speech is commonly judged by transcribing it with a recogniser [7] on a multilingual benchmark [8]; topic fit uses multilingual sentence embeddings [18] and ROUGE [19].

TABLE I
RELATED WORK COMPARED

Work	Approach	Data	Reported result	Limitation
SAM Translator [1]	mBART units + Grad-TTS, SPL, MoE	CVSS-C, CoVoST-2	BLEU +2-10% rel.; Welsh +21.6%	English target; A100, 1.5-2 days
OmniVoice [3]	Diffusion LM, zero-shot TTS	581k h, 646 languages	Mean CER 4.00%; RTF 0.0319	8 GPUs, 9.7 days; no story text
MultiGen [5]	LLM speech tokens, child voices	Private Malay, Tamil, Mandarin	Tamil CER 9.6% to 2.1%	3 languages; data not public
EmoSSLSphere [4]	Spherical emotion vectors	English, Japanese	Better prosody (listening test)	2 languages; no story content
Kahaani [17]	LLM story co-creation + speech	Not specified	Interactive system	No multilingual narration reported
MMS [2]	VITS checkpoint per language	1,107 languages	ASR halves Whisper WER (54 languages)	Single voice; narrow domain

Gap. The surveyed systems advance speech translation, massively multilingual synthesis and speech evaluation, but none joins controllable story generation to multilingual narration in one locally runnable application and then measures the whole chain with the story model, the speech model and a recogniser. Translation papers report BLEU and synthesis papers report MOS or CER on read sentences; neither says whether a small model on a laptop can tell a story in Hindi.

Positioning. StoryVox borrows rather than competes. From MMS it takes the per-language checkpoints, from Whisper the role of judge, and from the base paper the idea of scoring speech by transcribing it; it does not take the base paper's training recipe. The datasets it relies on are the pretrained models' own training corpora, which it does not touch, and FLEURS [8], a read-speech benchmark with the same sentences in 102 languages, which supplies the test text. The algorithms are standard: autoregressive sampling with a temperature and a nucleus cut-off [16], sentence chunking, a VITS forward pass, and edit-distance error rates. The strengths of this design are that it runs offline on modest hardware and that every stage can be inspected; the weaknesses are the small story model, one fixed voice per language, and error that accumulates along the chain.

III. METHODOLOGY

A. Pipeline and Design Choice

StoryVox is a cascade of three pretrained models behind one API (Fig. 1); no model was trained or fine-tuned. A cascade was chosen over a single end-to-end model because each stage can be replaced, each runs on a CPU and each can be measured separately, whereas the base paper's model needs a GPU and a training run that a student project cannot repeat. The speech stage sits behind an abstract provider interface with two implementations, MMS-TTS first and an operating-system voice second, so that a missing checkpoint degrades the result instead of stopping the request.

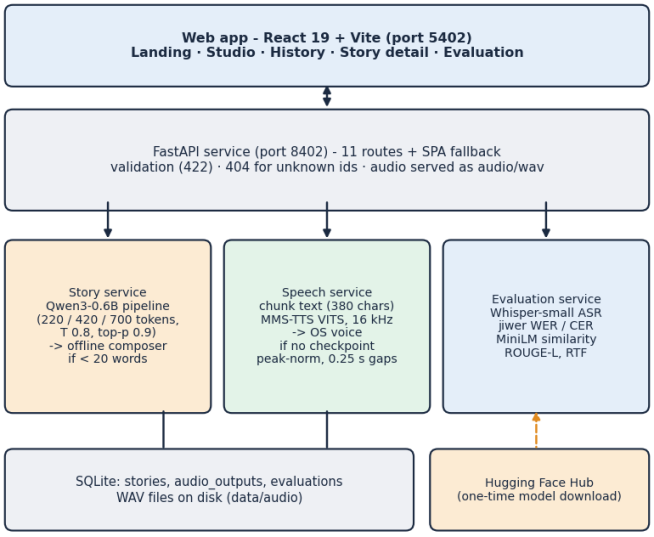


Fig. 1. System architecture of StoryVox: web app, FastAPI service, three model services and storage.

B. Story Generation

The prompt names the idea, the language, the style and the paragraph count (3, 5 or 8 for Short, Medium and Long) and asks for a title line followed by the story. It is sent unchanged, without the model's chat template, to a Transformers text-generation pipeline with Qwen3-0.6B, temperature 0.8, nucleus probability 0.9 and a budget of 220, 420 or 700 new tokens. The first line becomes the title. If the output has fewer than 20 words or the pipeline fails, an offline composer writes a template story that quotes the idea and says so in its model field. The eight styles (Adventure, Fantasy, Educational, Bedtime, Mystery, Inspirational, Funny, Custom) only change the style word and a short guidance phrase in the prompt; there is no separate model or adapter per style, and no reading-level control exists. At each step the model assigns a score z to every word piece; with temperature T the sampling distribution is

$$p_T(w) = \exp(z_w / T) / \sum_v \exp(z_v / T) \qquad (1)$$

and nucleus sampling [16] draws only from the smallest set of word pieces whose probabilities add up to 0.9. A lower T or a smaller nucleus makes the text more predictable and repetitive; T = 0.8 and 0.9 are the application's settings, and we did not tune them. Story generation is the expensive stage because it is autoregressive: each new token needs one forward pass of the 0.6B model, so time grows with the token budget, whereas the VITS decoder produces a whole chunk at once.

C. Narration

The text is split at sentence ends into chunks of at most 380 characters; each chunk is synthesised by the language's MMS-TTS checkpoint [2], a VITS model [13] that outputs a 16 kHz waveform. Chunks are peak-normalised to 0.9 and joined with 0.25 s of silence, and a silent result is rejected. Languages without a checkpoint use the OS voice. A VITS model is deterministic enough that one forward pass per chunk is used; the checkpoints are cached in memory after the first call, which is why the first story in a language is slower. Korean text must first be romanised with uroman. Speed is the real-time factor

$$RTF = t_{synth} / t_{audio} \qquad (2)$$

where t_synth is wall-clock synthesis time and t_audio the duration of the audio; values below 1 are faster than real time.

D. Evaluation Protocol

Round trip: ten sentences per language (6-30 words, 12-70 characters for Chinese and Japanese) are taken from the FLEURS test split [8] (CC BY 4.0), narrated by the application's own service and transcribed by Whisper-small with the language forced. Error rates use the usual definition

$$WER = (S + D + I) / N \qquad (3)$$

with S, D and I the substitutions, deletions and insertions against N reference words; CER is the same at character level. We report normalised scores (Unicode NFKC, lower case, punctuation removed, spaces removed for Chinese and Japanese, where WER is not reported) because the application's own function compares raw strings. The error rates combine the speech model's and the recogniser's errors; the recogniser's own floor was not measured.

Story study: one idea ("A young inventor discovers a hidden library."), Short length, seed 0, 17 cases (English in three styles and the other 14 languages in the Adventure style) under three conditions: A the shipped path; B the same prompt through the Qwen3 chat template with thinking disabled (an experiment, not in the product); C the offline composer. Metrics are generation latency, language match (langdetect), the share of sentences that repeat instructions, distinct bigrams, and multilingual MiniLM [18] cosine and ROUGE-L [19] between idea and story. These are proxies for on-topic, varied text; no person rated the stories.

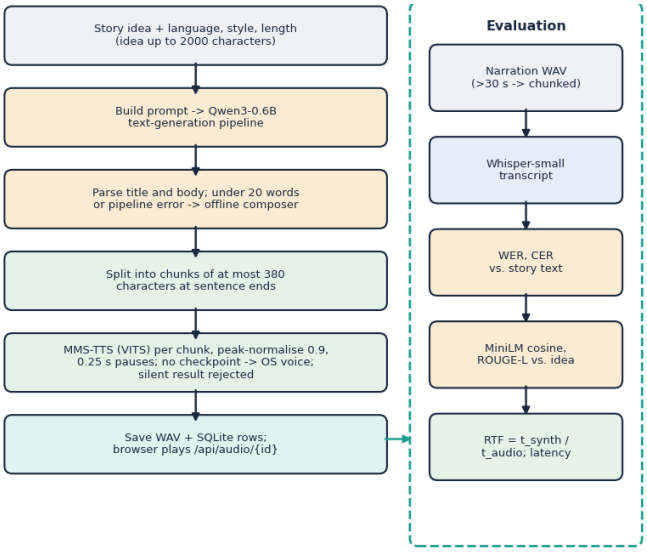


Fig. 2. Main flow: from story idea to saved narration (left) and the evaluation branch (right).

E. Implementation

The service has 11 routes, three SQLite tables (stories, audio_outputs, evaluations) and validates every request (empty or over-long prompt, unknown language, style or length return 422; unknown ids 404). The console has five pages. 29 automated tests pass (8 on the API and 21 on helpers, the composer in all 15 languages and edge cases); they stub the models, so the real models are exercised by the measurement scripts. Failures are reported instead of hidden: a silent waveform is rejected, a model that fails to load falls back to the next provider, and an unreadable audio path is refused (a test checks that a path escaping the audio folder is rejected). Deleting a story removes its audio file and its rows. The service is meant for one user on one machine: it has no accounts or authentication, SQLite foreign-key enforcement is not switched on, and the story model is loaded with remote-code execution enabled in the Transformers call, a setting a deployment should review.

IV. RESULTS

A. Test Conditions

CPU only, one run per case, seed 0 where sampling is used. Whisper-small loaded in 6.6 s and Qwen3-0.6B in 4.4 s; ten of the twelve MMS checkpoints took 19-25 s to load, English and Korean 1.8 s. Numbers from ten sentences per language carry wide uncertainty and should be read as indications.

B. Narration Speed and Intelligibility

Twelve languages are narrated by MMS (Fig. 3). The mean real-time factor is 0.247; eleven of them lie between 0.221 and 0.232 and Korean is slower at 0.46. Mean normalised CER is 0.088 and mean WER 0.250. Spanish is clearest (WER 0.082, CER 0.021) and Hindi worst (WER 0.462, CER 0.179); English reaches WER 0.206 and CER 0.091. WER is roughly two to four times CER in every language, as expected, because one wrong character makes a

whole word wrong. Because a round trip scores the speech model and Whisper-small together, a high Hindi error may come from either; we did not run the recogniser on natural Hindi speech to separate them.

TABLE II
MEASURED RESULTS (CPU, NORMALISED SCORES)

Measurement	Value
Narration, 12 MMS languages: mean RTF	0.247
Mean CER / WER after round trip	0.088 / 0.250
English WER / CER (10 sentences)	0.206 / 0.091
Italian (OS voice) WER / CER	0.494 / 0.144
Chinese / Japanese: sentences spoken, CER	2/10, 0.85 / 1/10, 1.00
Story latency, shipped path A (Short)	22.7 s (9.1 tok/s)
Language match A / B / C	0.18 / 0.47 / 0.18
Instruction repeats A / B / C	0.097 / 0.006 / 0.029
Distinct bigrams A / B / C	0.79 / 0.96 / 0.91
Idea-story similarity A / B / C	0.55 / 0.58 / 0.68
End to end, Short / Medium / Long	37 / 60 / 80 s
English story WER / CER (chunked ASR, n=3)	0.25 / 0.21
API median latency, non-model routes	1.2-15.0 ms

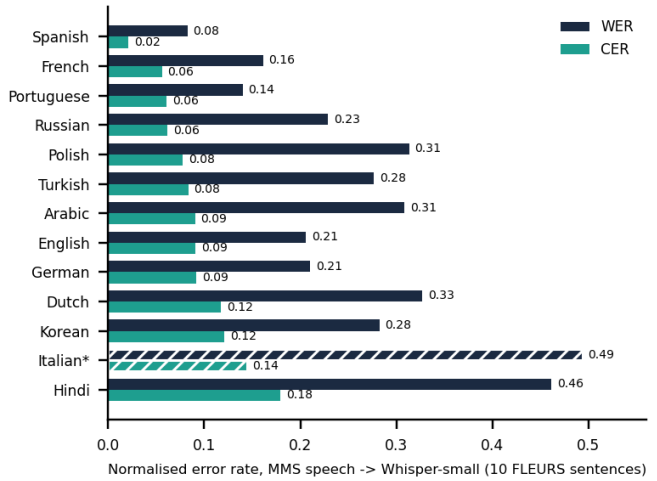


Fig. 3. Normalised WER and CER per language (10 FLEURS sentences each). Hatched bars (*) use the OS voice because no MMS checkpoint exists. Chinese and Japanese are omitted (CER 0.85 and 1.00).

Italian, Chinese and Japanese have no MMS checkpoint on the model hub (the checkpoint names the application requests do not exist on the hub), so the operating-system voice speaks them. On the test machine only English voices are installed: Italian is read with an English voice (WER 0.49), and the fallback produced audio for only 2 of 10 Chinese and 1 of 10 Japanese sentences, with CER 0.85 and 1.00. Korean works only after installing the romaniser uroman, which the requirements file does not list.

Speed is almost independent of language: ten of the twelve MMS languages lie within 0.22-0.23, because the VITS decoder cost depends on audio length, not on the script. Arabic sentences were the longest (13.7 s of audio on average against 8.9 s for English) yet kept the same factor. The Korean factor of 0.46 includes romanisation. At these factors a one-minute narration costs about 15 s of synthesis on the CPU, so playback can start after a short wait but not instantly.

C. Story Generation

Story latency is about 23 s for a Short story at 9.1 tokens/s and scales with the token budget (Table II, Fig. 7). With the shipped prompt the model wrote in the requested language in 0.18 of 17 cases, which are the three English cases; for every other language it produced the same English tale, often headed "Untitled Story", and 10% of its sentences repeated the instructions on average. With the chat template (B) the match rises to 0.47 and instruction repeats nearly vanish, but target-language text appears only for Spanish, German, Portuguese, Dutch, Chinese; Hindi, French, Italian, Russian, Arabic, Turkish, Polish, Japanese and Korean stay English. The composer is instant but template-like; its high idea-story similarity (0.68) comes from quoting the idea and is not a quality signal. Because condition A with seed 0 repeats near-identical text, its variety score is low.

Length scales roughly linearly with the token budget. A Short story (220 tokens) took 22 s to write and 15 s to narrate (175 words, 59 s of audio); Medium (420 tokens) 39 s and 21 s (316 words, 96 s); Long (700 tokens) 50 s and 30 s (418 words, 137 s). Writing therefore takes 60-65% of the end-to-end time. Over seven real calls through the API, the request took 23-67 s, the first (cold) call 42 s, while the routes that load no model answered in 1.2-15.0 ms at the median.

D. Story-Length Narration and the Evaluation Defect

Narration of a story lasts 44-137 s in our runs (Hindi aside). Whisper's pipeline call used by the application's evaluation rejects audio longer than 30 s under Transformers 5.20, so all five samples of its evaluation run carry an ASR error and the page shows WER and CER as dashes (Fig. 5). Scoring the same audio with 30 s chunking gives WER 0.25 and CER 0.21 for the three English stories. The four stories requested in Hindi, French, German and Spanish were English text read by that language's voice: WER was 0.60-1.34 for French, German and Spanish, and the Hindi voice produced 4.4 s of audio for 185 words. The fix is one argument; we did not change the product.

Failure cases seen in the saved stories: an English story that opens with the line "Now, write the story in the required format", a German request whose title came back as the instruction "The title should include ...", and a Short English story whose text states that it "should contain 22 characters, 1000 words" (visible in Fig. 4). Such lines are the model continuing the prompt instead of answering it, which the chat template largely removes.

E. Output Screens

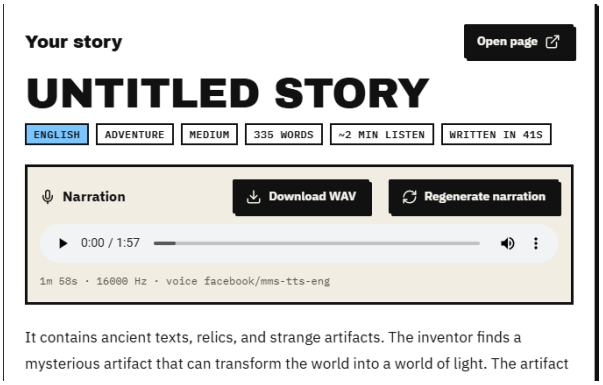


Fig. 4. Studio page with a generated English story, its narration player and the model that produced the voice (real capture, crop).

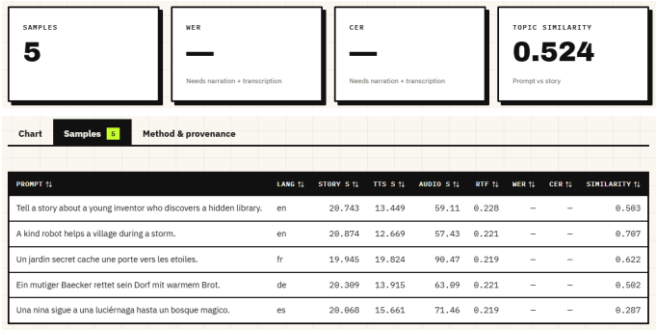


Fig. 5. Evaluation page (real capture; metric cards and Samples tab joined): speed, latency and similarity are measured; WER and CER are empty because the ASR step failed on audio over 30 s.

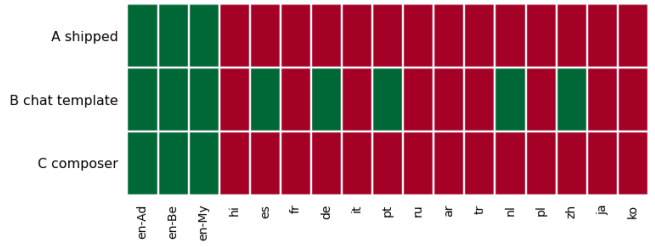


Fig. 6. Did the story come out in the requested language (green) or not (red)? 17 cases, three conditions; en-Ad, en-Be and en-My are English in the Adventure, Bedtime and Mystery styles.

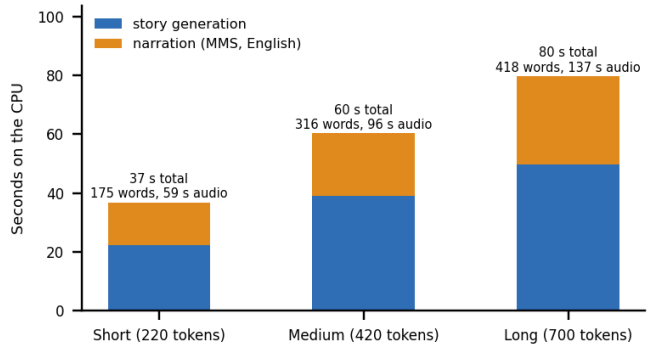


Fig. 7. End-to-end time by story length on the CPU: generation grows with the token budget; narration follows the audio length.

F. Comparison with the Base Paper

The scores cannot be compared. SAM Translator [1] reports BLEU for translation into English speech on CVSS-C and CoVoST-2 after training on a GPU; StoryVox translates nothing, trains nothing and reports round-trip error rates and speed on FLEURS sentences. OmniVoice's 4.00% CER [3] was measured on a GPU over 102 languages; our CER is not comparable either, since the recogniser, the sample (ten sentences) and the hardware all differ. What can be compared is the design: a cascade of small pretrained parts that runs on a CPU in about 37 s for a Short story versus a trained model with 1.7 billion parameters.

G. Reproducibility

All figures in this section are read from JSON files written by four scripts in the repository (round-trip and story experiments, API timing, test run, and chunked story scoring); each script calls the application's own service functions, so the numbers describe the shipped behaviour rather than a re-implementation. The Hugging Face hub supplies FLEURS and all weights, and the machine and library versions are given in Section I.

H. Objectives against Outcomes

The batch objectives were: (1) accept a prompt, met; (2) generate a story with length and reading-level controls, partly met (three lengths and eight styles, no reading level); (3) translate into target languages, not met, since the language is only requested in the prompt and the small model usually ignores it; (4) convert to speech, met for twelve languages and unreliable for Italian, Chinese and Japanese; (5) play back with adjustable voice, speed and language, partly met (a browser player with seek and download, language fixed at generation, no voice or speed choice); (6) evaluate coherence, translation quality and naturalness through user rating, partly met (automatic proxies and a recogniser-based score, no ratings from people).

I. Limits

Threats to validity: language identification with langdetect is unreliable on short text; the idea-story similarity rewards copying; one seed gives one sample of each condition; the chat-template condition was not applied to the product. Ten sentences per language, one machine, one idea and one seed; no human listeners; the recogniser's error floor unmeasured; story quality judged only by proxies. Translation, voice or speed selection and user ratings, which the project objectives list, are not implemented.

V. CONCLUSION AND FUTURE SCOPE

StoryVox shows that a cascade of small pretrained models can turn an idea into a stored narration on a CPU in 37-80 s, and that narration is the strong stage: twelve languages are read at about a quarter of real time with a mean normalised CER of 0.088. The main finding is negative: the 0.6B story model, prompted as shipped, writes English whatever language is requested, so the speech model then reads text it was not built for. Of the six objectives, prompt input and speech output are met, story length and style controls partly, and translation, voice and speed choice and user rating not at all. Four changes would help most: apply the chat template and check the language of the output,

chunk audio in the evaluation, add the missing checkpoints and dependencies to the requirements, and add a translation step or a larger model. Listening tests with people would turn the proxy scores into evidence of quality.

Future scope in order of expected value: (1) a language check on the generated text with an automatic retry or a translation fallback; (2) a larger or multilingual-tuned story model, or a two-step design that writes in English and translates with a dedicated model; (3) voice and speed selection, and per-language checkpoints beyond MMS for Italian, Chinese and Japanese; (4) a human rating form in the console, which would also meet the user-rating objective; (5) measuring the recogniser's own error on natural speech so that the speech model's share of each error rate can be separated; (6) a GPU or quantised build to bring a Short story well under half a minute.

REFERENCES

[1] C. Tran and S. Sakti, "SAM Translator: Self-paced learning and mixture-of-experts for cross-lingual text-to-speech translation," IEEE Access, vol. 14, pp. 2742-2752, 2026, doi: 10.1109/ACCESS.2025.3644647.

[2] V. Pratap et al., "Scaling speech technology to 1,000+ languages," Journal of Machine Learning Research, vol. 25, no. 97, pp. 1-52, 2024.

[3] H. Zhu, L. Ye, W. Kang, Z. Yao, L. Guo, F. Kuang, Z. Han, W. Zhuang, L. Lin and D. Povey, "OmniVoice: Towards omnilingual zero-shot text-to-speech with diffusion language models," arXiv:2604.00688, Apr. 2026.

[4] J. Park and K. Nakamura, "EmoSSLSphere: Multilingual emotional speech synthesis with spherical vectors and discrete speech tokens," in Proc. 13th ISCA Speech Synthesis Workshop (SSW), 2025, arXiv:2508.11273.

[5] X. Gao, H. Zhang and N. F. Chen, "MultiGen: Child-friendly multilingual speech generator with LLMs," in Companion Proc. 27th Int. Conf. Multimodal Interaction (ICMI Companion '25), Canberra, Australia, Oct. 2025, doi: 10.1145/3747327.3764897.

[6] A. Yang et al., "Qwen3 technical report," arXiv:2505.09388, May 2025.

[7] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in Proc. 40th Int. Conf. Machine Learning (ICML), PMLR vol. 202, 2023, pp. 28492-28518.

[8] A. Conneau, M. Ma, S. Khanuja, Y. Zhang, V. Axelrod, S. Dalmia, J. Riesa, C. Rivera and A. Bapna, "FLEURS: Few-shot learning evaluation of universal representations of speech," in Proc. IEEE Spoken Language Technology Workshop (SLT), 2022, arXiv:2205.12446.

[9] T. Wolf et al., "Transformers: State-of-the-art natural language processing," in Proc. Conf. Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP), 2020, pp. 38-45.

[10] Y. Liu, J. Gu, N. Goyal, X. Li, S. Edunov, M. Ghazvininejad, M. Lewis and L. Zettlemoyer, "Multilingual denoising pre-training for neural machine translation," Trans. Association for Computational Linguistics, vol. 8, pp. 726-742, 2020.

[11] M. P. Kumar, B. Packer and D. Koller, "Self-paced learning for latent variable models," in Proc. Advances in Neural Information Processing Systems (NIPS), vol. 23, 2010, pp. 1189-1197.

[12] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in Proc. 5th Int. Conf. Learning Representations (ICLR), 2017.

[13] J. Kim, J. Kong and J. Son, "Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech," in Proc. 38th Int. Conf. Machine Learning (ICML), PMLR vol. 139, Jul. 2021, pp. 5530-5540.

[14] R. Eldan and Y. Li, "TinyStories: How small can language models be and still speak coherent English?," arXiv:2305.07759, 2023.

[15] A. Fan, M. Lewis and Y. Dauphin, "Hierarchical neural story generation," in Proc. 56th Annual Meeting of the Association for Computational Linguistics (ACL), 2018, pp. 889-898.

[16] A. Holtzman, J. Buys, L. Du, M. Forbes and Y. Choi, "The curious case of neural text degeneration," in Proc. 8th Int. Conf. Learning Representations (ICLR), 2020.

[17] S. Arif, M. S. Haroon, A. J. Khan, T. Arif, A. A. Raza and A. Athar, "Kahaani: A multimodal co-creative storytelling system," arXiv:2409.11261, 2024 (rev. Feb. 2026).

[18] N. Reimers and I. Gurevych, "Making monolingual sentence embeddings multilingual using knowledge distillation," in Proc. Conf. Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 4512-4525.

[19] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in Text Summarization Branches Out, Proc. ACL-04 Workshop, Barcelona, Spain, 2004, pp. 74-81.